🧠 DSA Notes

Loop Analysis Patterns — Beginner's Cheatsheet

Each card below shows a loop pattern, its code, and all three notations: Best (Ω), Worst (O), and Theta (Θ).

📌 1. Single Loop

```
for (let i = 0; i < n; i++) {
  // runs n times
}
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n)	O(n)	Θ(n)

👉 Always runs exactly n times. No early exit. Best = Worst → Theta defined.

📌 2. Nested Loop (Full)

```
for (let i = 0; i < n; i++) {
  for (let j = 0; j < n; j++) {
    // inner runs n times per outer step
  }
}
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n²)	O(n²)	Θ(n²)

👉 Outer runs n times, inner runs n times per outer → n × n = n². Always.

📌 3. Increasing Pattern (Triangle)

```
for (let i = 1; i <= n; i++) {
  for (let j = 1; j <= i; j++) {}  // j goes up to i
}

// Steps: 1 + 2 + 3 + ... + n = n(n+1)/2  →  O(n²)
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n²)	O(n²)	Θ(n²)

👉 Inner grows with outer: 1+2+…+n = n(n+1)/2. Drop constant → n². Always same.

📌 4. Decreasing Pattern

```
for (let i = 1; i <= n; i++) {
  for (let j = i; j <= n; j++) {}  // j starts from i
}

// Steps: n + (n-1) + ... + 1 = n(n+1)/2  →  O(n²)
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n²)	O(n²)	Θ(n²)

👉 Mirror of increasing: n+(n-1)+…+1 = same formula → n². Always the same.

📌 5. Logarithmic Loop — Doubling

```
for (let i = 1; i <= n; i *= 2) {}

// i: 1 → 2 → 4 → 8 → 16 → ... → n
// Takes log₂(n) steps to reach n
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(log n)	O(log n)	Θ(log n)

👉 i doubles every step → takes $\log_2(n)$ steps. Binary search uses this pattern.

📌 6. Logarithmic Loop — Halving

```
for (let i = n; i > 1; i /= 2) {}

// i: n → n/2 → n/4 → ... → 1
// Takes log₂(n) steps to reach 1
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(log n)	O(log n)	Θ(log n)

👉 i halves every step → same $\log_2(n)$ steps, just reversed direction.

📌 7. Mixed Nested (log × n)

```
for (let i = 1; i <= n; i *= 2) {    // outer: log n steps
  for (let j = 0; j < n; j++) {      // inner: n steps each
    // ...
  }
}

// log n × n = n log n
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n log n)	O(n log n)	Θ(n log n)

👉 Outer runs log n times. Each time, inner runs n steps. Nested → multiply.

📌 8. Separate Loops

```
for (let i = 0; i < n; i++) {}   // O(n)
for (let j = 0; j < n; j++) {}   // O(n)
```

```
// Separate, not nested → ADD: O(n) + O(n) = O(2n) → O(n)
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n)	O(n)	Θ(n)

👉 Sequential loops add up. O(2n) simplifies to O(n) — constants are ignored.

📌 9. Early Exit (break / return)

```
for (let i = 0; i < n; i++) {
  if (condition) break;   // might exit on first step!
}
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(1)	O(n)	❌ Not defined

👉 May exit on step 1 (best) or run all n steps (worst). Best ≠ Worst → no Theta.

📌 10. Conditional + Inner Loop

```
for (let i = 0; i < n; i++) {
  if (condition) {
    for (let j = 0; j < n; j++) {}
    break;    // inner runs once, then outer breaks
  }
}
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(n)	O(n)	❌ Not defined

👉 Inner loop runs only once. Outer scans up to n to find the condition. Theta undefined — depends on where condition triggers.

📌 11. If Condition Inside Loop (Search)

```
for (let i = 0; i < n; i++) {
  if (arr[i] === target) return true;  // exit when found
}
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(1)	O(n)	❌ Not defined

> 👉 Best: target at index 0 → 1 step. Worst: target at end or missing → n steps.

📌 12. Always Constant (No Loop)

```
let x = 10;    // constant operation
return x;      // constant operation

// No n → no growth → always O(1)
```

Best (Ω)	Worst (O)	Theta (Θ)
Ω(1)	O(1)	Θ(1)

> 👉 No dependency on input size n. Always executes in fixed number of steps.

📊 Quick Reference — All Patterns at a Glance

Pattern	Best (Ω)	Worst (O)	Theta (Θ)
Single loop	Ω(n)	O(n)	Θ(n)
Nested loop	Ω(n²)	O(n²)	Θ(n²)
Increasing pattern	Ω(n²)	O(n²)	Θ(n²)
Decreasing pattern	Ω(n²)	O(n²)	Θ(n²)
Doubling (i *= 2)	Ω(log n)	O(log n)	Θ(log n)
Halving (i /= 2)	Ω(log n)	O(log n)	Θ(log n)
Mixed (log × n)	Ω(n log n)	O(n log n)	Θ(n log n)

Separate loops	`Ω(n)`	`O(n)`	`Θ(n)`
Early exit (break)	`Ω(1)`	`O(n)`	❌ None
Conditional + inner	`Ω(n)`	`O(n)`	❌ None
If + return in loop	`Ω(1)`	`O(n)`	❌ None
Constant (no loop)	`Ω(1)`	`O(1)`	`Θ(1)`

🔥 Key Rules

Rule 1: Add vs Multiply

Separate loops (one after another) → ADD their complexities
Nested loops (one inside another) → MULTIPLY their complexities

Rule 2: Early Exit Pattern

Best case → Ω(1) (exits on first match)
Worst case → O(n) (runs through all elements)
Theta → ❌ Not defined (behavior varies by input)

Rule 3: Theta Condition

Theta (Θ) is only defined when: Best case = Worst case
If any condition can cause early exit → Theta is undefined

Rule 4: Logarithmic Pattern

i *= 2 (doubling) → O(log n) — loop runs $\log_2(n)$ times
i /= 2 (halving) → O(log n) — same, just in reverse

🎯 Final Takeaway

🎯 Always Ask These Questions

- Does it always run the same number of steps? → **Use Θ (theta)**
- Can it exit early via break/return? → **Best is Ω(1), Theta undefined**
- Are loops nested (one inside another)? → **Multiply complexities**
- Are loops separate (one after another)? → **Add complexities**

- Does i double or halve each step? → **O(log n)**